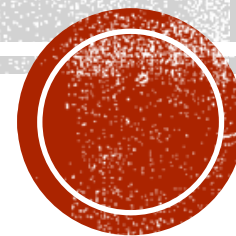


BANCOS DE DADOS DE CHAVE- VALOR



BANCOS DE DADOS DE CHAVE-VALOR

- Nesse tipo de banco, o agregado (o valor) é opaco (apenas um grande amontoado de bits). Diferentemente de um agregado do tipo documento, onde é possível a existência de uma estrutura;
- A vantagem desta opacidade é a possibilidade de armazenarmos o que quisermos no agregado;
- O banco pode impor algum limite de tamanho mas, de forma geral, temos liberdade completa quanto aos dados a serem persistidos;
- A desvantagem gerada por essa liberdade é a queda na flexibilidade de acesso aos dados, já que as consultas só podem ser feitas a partir de chaves;



BANCOS DE DADOS DE CHAVE-VALOR

- Estudo de caso: **Riak**
 - Permite alta disponibilidade de dados;
 - Possui um modelo de replicação de dados avançado;
 - Projeto de código aberto escrito em Erlang;
 - Permite a manipulação de dados a partir de HTTP (REST);
 - Distribuição gratuita para Linux e MacOS;



RIAK

- Comparativo entre conceitos do modelo relacional e do MongoDB:

Conceito relacional	Conceito Riak
Database	
Tabela	Bucket
Linha	Objeto
Coluna	Chave-valor
Index	Chave
Table Join	
Chave primária	



RIAK

- Criando nosso primeiro objeto:
 - O Riak fornece um cliente chamado **riak console**, que permite executar comandos para a manipulação de dados. Para executá-lo, digite o seguinte comando em um terminal:

```
$ sudo riak console
```

- Para podermos manipular dados, vamos precisar configurar em qual servidor nos conectaremos. Primeiramente, iremos criar um objeto que represente o servidor e sua localização, a partir do comando abaixo:

```
(riak@127.0.0.1)1> Servidor = `riak@127.0.0.1'.
```



RIAK

- Criando nosso primeiro objeto:
 - Para verificar se o servidor está ativo, podemos executar o comando opcional **ping** a partir do objeto **net_adm** (se estiver ok, responderá com **pong**)

```
(riak@127.0.0.1)2> net_adm:ping(Servidor) .
```

- Para criar uma conexão com o servidor, utilizaremos a função **client_connect** a partir do objeto **riak**:

```
(riak@127.0.0.1)3> {ok, Conexao} = riak:client_connect(Servidor) .
```



RIAK

- Criando nosso primeiro objeto:
 - Para criar um objeto, precisamos indicar o bucket que delimitará os dados, além dos valores de chave-valor que irá possuir. Para isso utilizamos o comando **new** do objeto **riak_object**. Exemplo:

```
(riak@127.0.0.1)4> Voto =  
riak_object:new(<<"votacao">>, <<"davidpaniz">>, <<"Fulano">>).
```

- Para registrar o objeto no servidor, utilizaremos a função **put** a partir do objeto que representa a conexão com o mesmo

```
(riak@127.0.0.1)5> Conexao:put(Voto, 1).
```



RIAK

Similar a consulta SQL: `INSERT INTO votacao (chave, valor) VALUES ("davidpaniz", "Fulano");`

- Criando nosso primeiro objeto:
 - Para criar um objeto, precisamos indicar o bucket que delimitará os dados, além dos valores de chave-valor que irá possuir. Para isso utilizamos o comando **new** do objeto **riak_object**. Exemplo:

```
(riak@127.0.0.1)4> Voto =  
riak_object:new(<<"votacao">>, <<"davidpaniz">>, <<"Fulano">>).
```

- Para registrar o objeto no servidor, utilizaremos a função **put** a partir do objeto que representa a conexão com o mesmo

```
(riak@127.0.0.1)5> Conexao:put(Voto, 1).
```



RIAK

- Até o momento, utilizamos Erlang para realizar uma consulta no Riak. Também é possível utilizar **requisições HTTP** para realiza-las;
- O Linux dispõe do comando **curl** para que possamos gerar requisições HTTP a partir de um terminal;
- Exemplo: Vamos fazer uma busca no banco, resgatando o voto do usuário davidpaniz a partir de HTTP:

```
$ curl http://localhost:8098/types/default/buckets/votacao/keys/davidpaniz  
Fulano
```



RIAK

Similar a consulta SQL: `SELECT valor FROM votacao WHERE chave = "davidpaniz";`

- Até o momento, utilizamos Erlang para realizar uma consulta no Riak. Também é possível utilizar **requisições HTTP** para realiza-las;
- O Linux dispõe do comando **curl** para que possamos gerar requisições HTTP a partir de um terminal;
- Exemplo: Vamos fazer uma busca no banco, resgatando o voto do usuário davidpaniz a partir de HTTP:

```
$ curl http://localhost:8098/types/default/buckets/votacao/keys/davidpaniz  
Fulano
```



RIAK

- Assim como é possível ler dados do banco a partir de uma chave através de HTTP, também é possível criar uma chave nova;
- Exemplo: Vamos fazer com que o usuário **adriano** vote em **Ciclano** a partir de HTTP:

```
$ curl -XPUT -d "Ciclano"  
http://localhost:8098/types/default/buckets/votacao/keys/adriano
```

- Exemplo: Agora vamos obter o voto de adriano:

```
$ curl http://localhost:8098/types/default/buckets/votacao/keys/adriano  
Ciclano
```



RIAK

- Também é possível obter todas as chaves cadastradas via HTTP:

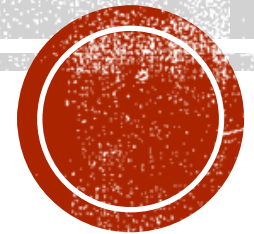
```
$ curl http://localhost:8098/types/default/buckets/votacao/keys?keys=true
```

- Além de excluir registros. Exemplo: Excluir o voto de adriano:

```
$ curl -XDELETE  
http://localhost:8098/types/default/buckets/votacao/keys/adriano
```



ATIVIDADE 2



ATIVIDADE 2

- Etapa 1: Via HTTP, insira os 6 objetos chave-valor listados a seguir em três buckets diferentes:

Chave (nome)	Valor (idade)	Bucket
Thyago	35	professores
Afonso	28	professores
Fernanda	32	alunos
Theo	12	alunos
Sophia	20	funcionarios
Leonardo	15	funcionarios

- Liste todas as chaves de cada bucket separadamente;

- Entregável: No VirtualBox, clique no menu **Visualizar / Capturar Tela**. Salve uma imagem da tela após executar as consultas corretamente. Entregue tal imagem como comprovante de conclusão dessa etapa;



ATIVIDADE 2

- Etapa 2:
 - Altere a idade de Theo para 25;
 - Faça com que Leonardo deixe de ser um funcionário e passe a ser um professor;
 - Exclua Afonso;
- Obtenha a nova idade de Theo e apresente no console; Liste as chaves de cada bucket separadamente;
- Entregável: No VirtualBox, clique no menu **Visualizar / Capturar Tela**. Salve uma imagem da tela após executar as consultas corretamente. Entregue tal imagem como comprovante de conclusão dessa etapa;

